ACTUALIZACION DE ARQUITECTURA DE LA IMPLEMENTACION

SushiNice V7 | Anexo tecnico vigente

Esta hoja corrige y reemplaza las descripciones tecnicas de arquitectura en las paginas originales de este informe. El diseno descrito originalmente con Node.js, Express y JWT no corresponde al software entregado.

Stack implementado

PHP 8 en Apache, acceso a MySQL mediante PDO y una interfaz HTML, CSS y JavaScript. El acceso a cuentas usa sesiones PHP en auth.php; no existe un backend Express ni autenticacion JWT en esta entrega.

Persistencia

database.sql crea usuarios, menu_items, pedidos, pedido_items y mensajes_contacto, y carga los 24 productos iniciales. api.php guarda pedidos de invitados con precios recalculados en el servidor, mensajes de contacto, cambios de menu y estados de pedidos. El panel calcula ventas y productos mas pedidos desde esos registros.

Acceso administrativo

Los endpoints del panel verifican la sesion y el rol es_admin. El registro publico crea cuentas normales; un operador habilitado establece el rol administrativo directamente en MySQL.

Configuracion

La conexion lee DB_HOST, DB_NAME, DB_USER y DB_PASSWORD. Consulta README.md para los pasos de XAMPP, importacion del esquema y habilitacion segura de la cuenta administradora.

Alcance

Los datos de ventas mostrados son historicos y provienen de pedidos persistidos; no se presenta una prediccion simulada. La confirmacion registra el pedido, pero esta entrega no integra pasarela de pago, control de inventario ni seguimiento de envios.

La implementacion PHP/MySQL y README.md son la referencia vigente para operar y mantener este proyecto.

Universidad Autónoma del Estado de Hidalgo

Escuela Superior de Tlahuelilpan

Descubre la automatización de una cadena mejorar la experiencia del usuario Sushi Nice

SUSHI NICE

Autores:

Alondra Bautista Flores
Amelie Leocadio Nolasco
Zuly Meyai Martínez Velázquez Leonardo
Damián Hernández Hernández Jonathan
Flores Cordero

Noviembre, 2025

Contenido

Índice de Tablas 3

Índice de Figuras 3

Resumen 4

Abstract 4

Introducción 5

1. 6
   1.1. **¡Error! Marcador no definido.**
   1.2. 6
2. 7
   2.1. 7
   2.2. 7
   2.3. 8
   2.4. 9
3. 10
   3.1. 10
   3.2. 11
   3.3. 11
4. 12
   4.1. 13
   4.2. 13
5. 14
   5.1. 14
   5.2. 14
6. 15
7. 16
   7.1. 16
   7.2. 16
   7.3. 17
8. 18
   8.1. 18
   8.2. 18

9. 19
9.1. 19
9.2. 19
9.3. 20
9.4. 20
10. 21
10.1. 21
10.2. 21
11. 22
Conclusiones 12
Referencias Bibliográficas (formato APA). 13
Anexo A: Manual de Usuario 14

Índice de Tablas

Índice de Figuras

Resumen

l restaurante de sushi enfrenta una crisis operativa derivada de la gestión manual de comandas, lo que provoca discrepancias financieras y mermas de inventario. El proyecto "Sushi Nice" desarrolla un sistema Punto de Venta (POS) Inteligente basado en una arquitectura cliente-servidor, empleando HTML5, CSS3 y JavaScript en el *frontend* y Node.js con Express en el *backend*. El sistema centraliza la operativa mediante una API REST, implementando autenticación basada en JSON Web Tokens (JWT) para gestionar roles y asegurar la integridad de los datos en MySQL/MongoDB. El objetivo principal es eliminar errores de cobro, controlar el inventario en tiempo real y fomentar la lealtad del cliente mediante descuentos automatizados.

Abstract

A sushi restaurant faces financial and inventory losses due to the lack of control in the order cycle and recurring billing errors. The "Sushi Nice" project proposes an Intelligent Point of Sale (POS) system developed with a modern stack including HTML5, CSS3, JavaScript for the user interface, and Node.js with Express for the server-side, utilizing a REST API architecture. Security is ensured through JSON Web Tokens (JWT) for role-based authentication, while data is managed via MySQL or MongoDB. The general objective is to centralize operational management, eliminate billing errors, and improve customer loyalty through real-time inventory tracking and data intelligence.

Introducción

La industria restaurantera actual exige una eficiencia operativa superior para mantenerse competitiva. En el caso específico de "Sushi Nice", el modelo de gestión manual de comandas ha derivado en discrepancias financieras, mermas de inventario y una experiencia inconsistente para el cliente. Estos procesos, altamente dependientes de la intervención humana, son propensos a errores que comprometen la rentabilidad y la calidad del servicio.

Este proyecto propone el desarrollo de un Punto de Venta (POS) Inteligente que centraliza la gestión operativa. La solución aprovecha un *stack* tecnológico moderno: HTML5, CSS3 y JavaScript para una interfaz intuitiva, soportado por un *backend* escalable en Node.js con Express. La integración de una API REST permite la trazabilidad total de las operaciones, mientras que la autenticación mediante JSON Web Tokens (JWT) garantiza la seguridad y la correcta asignación de roles dentro del equipo.

A lo largo de este documento, se detalla la investigación preliminar, la metodología de desarrollo en cascada y la arquitectura del sistema. Con esta propuesta, "Sushi Nice" busca no solo automatizar el ciclo de la comanda, sino transformar su operativa mediante la inteligencia de datos, asegurando un control riguroso, mayor lealtad del cliente y una competitividad sostenida en un mercado altamente exigente.

.

1. Investigación preliminar

1.1 El restaurante de sushi enfrenta una crisis operativa y de gestión de clientes que impacta directamente su rentabilidad y potencial de crecimiento.

Descripción del problema lo más detallado y claro posible.

1.2 El proyecto para desarrollar un Punto de Venta Inteligente es factible desde varias perspectivas:

• Factibilidad Operativa: El personal está actualmente sufriendo las consecuencias del desorden (pérdida de dinero, quejas). La implementación de un sistema que fuerza la especificación detallada y el registro obligatorio de cada artículo simplificará sus tareas y eliminará los errores humanos, por lo que la aceptación será alta.

• Factibilidad Técnica: Hoy en día existen numerosas plataformas y tecnologías robustas (bases de datos SQL, lenguajes de programación web/móvil) que permiten desarrollar un POS con las funcionalidades requeridas (control de inventario, inteligencia de negocios y hardware simple como tabletas y terminales de punto de venta).

1.2.1 Aunque la inversión inicial en desarrollo será significativa, se espera un retorno de la inversión rápido al reducir las pérdidas por omisiones de cobro, mejorar el control de inventario (reduciendo el desperdicio) y, fundamentalmente, aumentar las ventas a través de la fidelización y las recomendaciones personalizadas. Las ganancias por el incremento de la lealtad superarán el costo del sistema.

2. Definición general del proyecto

2.1. Objetivo General

Desarrollar e implementar un Punto de Venta Inteligente para el restaurante de sushi que centralice la gestión operativa, elimine las pérdidas por errores en el cobro y las comandas, y utilice la inteligencia de datos para fidelizar a los clientes, impulsando así la eficiencia y competitividad del negocio

2.2. Objetivos Específicos

1. Garantizar el cobro íntegro de todos los productos consumidos al hacer obligatorio el registro de la comanda antes del cierre de cuenta.

2. Establecer un control preciso de inventario en tiempo real mediante la vinculación de cada venta con la receta exacta del platillo consumido.

3. Permitir el registro de especificaciones detalladas en las comandas (ej. frito/natural, con/sin picante, tipo de contenedor) para asegurar la calidad del producto y el uso correcto de insumos.

4. Implementar un módulo de fidelización que identifique a los clientes y aplique descuentos automáticos en compras mayores a $500.

5. Desarrollar un sistema de análisis de datos capaz de generar reportes de venta y ofrecer recomendaciones personalizadas al personal de servicio.

2.3. Metodología de desarrollo

Metodología de Desarrollo en Cascada

La metodología de desarrollo en cascada es un enfoque secuencial y lineal que consta de varias fases:

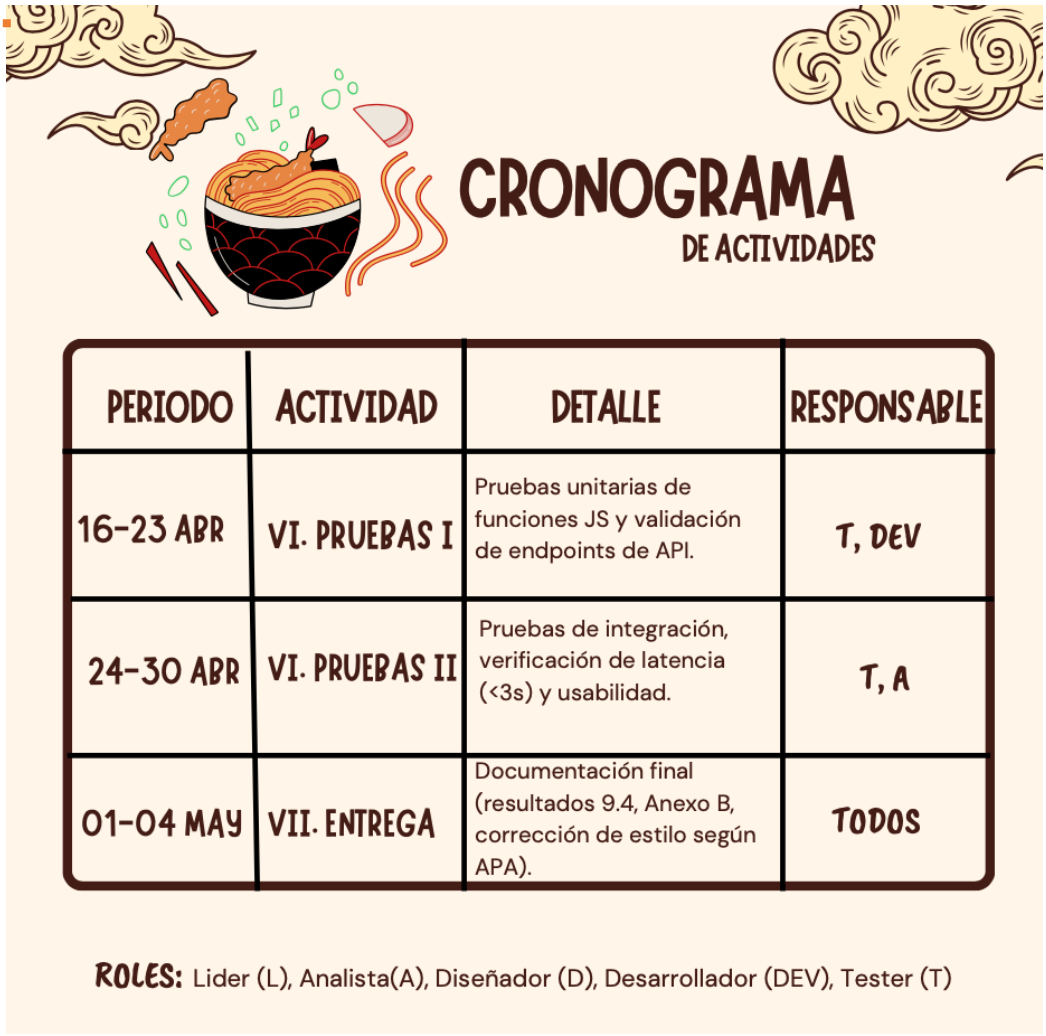
Requisitos: Se recopilan y documentan todos los requisitos del sistema. 2. Diseño: Se elabora un diseño técnico basado en los requisitos.

1. Implementación: Se desarrolla el software según el diseño.
2. Pruebas: Se realizan pruebas para identificar y corregir errores.
3. Despliegue: Se lanza el software en el entorno de producción.
4. Mantenimiento: Se proporciona soporte y actualizaciones.
5. Ventajas: Estructura clara, fácil gestión y documentación exhaustiva.
6. Desventajas: Rigidez ante cambios, alto costo de correcciones tardías y poco enfoque en retroalimentación continua.

2.4. Cronograma de actividades



PERIODO	ACTIVIDAD	DETALLE	RESPONSABLE
01-15 FEB	I. ANÁLISIS	Levantamiento de requerimientos y definición del modelo E-R (MySQL/MongoDB).	L, A
16-28 FEB	II. DISEÑO	Prototipado en Figma: flujo de interfaz táctil (máximo 3 clics). Diseño del esquema de API REST.	L, D
01-08 MAR	III. BACKEND I	Configuración de servidor Node.js/Express y conexión a base de datos.	DEV
09-15 MAR	III. BACKEND II	Creación de rutas de API REST (CRUD de productos, comandas e inventario).	DEV
16-25 MAR	IV. SEGURIDAD	Implementación de autenticación JWT y middleware para roles (Mesero, Cajero, Gerente).	DEV, L
26 MAR - 15 ABR	V. FRONTEND	Desarrollo de vistas HTML5/CSS3. Consumo de API REST mediante JavaScript (Fetch/Axios).	DEV, D



3. Especificación de requerimientos

3.1. Requerimientos funcionales

1.- Gestión de Órdenes (Comandas) Detallada:

El sistema debe permitir al mesero/cajero registrar una orden especificando todos los detalles del producto (e.g., tipo de rollo: frito/natural, ingredientes adicionales/excluidos, nivel de picante, tipo de contenedor si es para llevar, etc.).

2.- Registro y Modificación de Cuentas:

El sistema debe obligar a registrar todos los productos ordenados antes de generar la cuenta. Debe permitir agregar o modificar productos en la cuenta solo con autorización (e.g., clave de gerente).

4.- Identificación y Fidelización de Clientes

El sistema debe identificar a un cliente (por nombre, teléfono o código) al iniciar una orden y registrar su historial de consumo y preferencias (platillos favoritos, restricciones).

3.2. Requerimientos no funcionales

1.-Rendimiento: La generación de una comanda o la consulta de inventario no debe tardar más de 3 segundos para garantizar un servicio ágil.

2.- Fiabilidad y Disponibilidad: El sistema debe estar operativo el 99.5% del tiempo durante el horario de atención del restaurante para no interrumpir las operaciones.

3.- Seguridad: Se implementarán roles de usuario (Gerente, Cajero, Mesero) con permisos

restringidos. Solo el gerente podrá realizar acciones críticas como anular ventas o modificar inventario.

4.- Usabilidad (Interfaz Intuitiva): La interfaz de toma de pedidos debe ser táctil, clara y fácil de usar, permitiendo al mesero seleccionar las especificaciones detalladas (picante, frito, etc.) con un máximo de tres toques/clics por platillo.

5.- Compatibilidad: El sistema debe funcionar en tabletas Android (para el uso de meseros) y en terminales de escritorio (para el área de caja).

6.- Integridad de Datos: Todas las transacciones de venta, cobro y ajuste de inventario deben quedar registradas y auditables con la fecha, hora y el usuario que las realizó.

3.3. Técnicas y herramientas para la obtención, análisis, priorización y validación de requerimientos

Obtención

• Entrevistas Estructuradas: Se realizarán a gerentes, meseros y cocineros para entender los flujos de trabajo actuales y recabar los detalles finos de las especificaciones de las comandas.

• Observación de Procesos: Observar directamente cómo se toman las órdenes y se realizan los cobros para identificar exactamente dónde ocurren las omisiones de cobro y los errores de especificación.

• Análisis de Documentos: Se revisarán el menú y las recetas para determinar las reglas de negocio exactas y los componentes de inventario que debe descontar el sistema.

Análisis

• Diagramas de Caso de Uso: Se modelarán las interacciones principales del sistema, como "Gestionar Comanda Detallada" o "Aplicar Descuento Automático", para definir el alcance del sistema.

• Prototipos de Pantallas: Se crearán bocetos de las interfaces (caja y mesero) para que los usuarios puedan visualizar y validar la facilidad de uso y el proceso de registro detallado.

Priorización

• Método MoSCoW: Los requerimientos se clasificarán en: (Debe tener, ej. registro obligatorio de cuentas), (Debería tener, ej. control de inventario) y (Podría tener, ej. sugerencias personalizadas), enfocando los recursos en lo más crítico para el negocio.

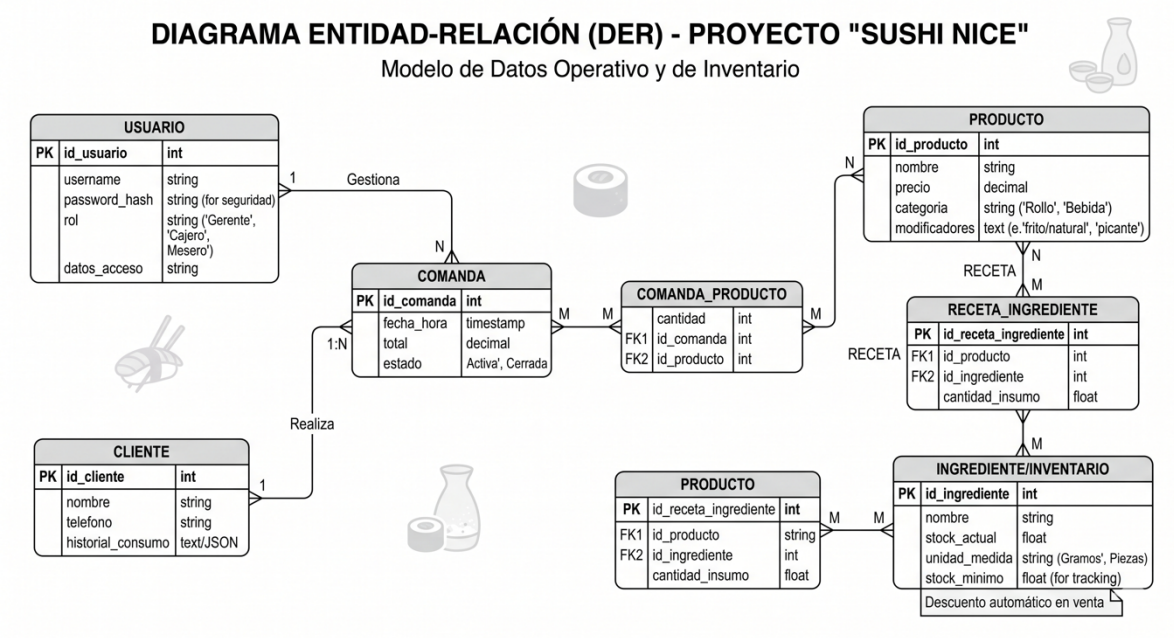
Validación

• Revisión Formal de Requerimientos: El documento final se presentará al dueño del restaurante y usuarios clave para su aprobación formal, asegurando que todos los requerimientos sean correctos.

• Pruebas Alfa y Beta: Se lanzará una versión preliminar para que meseros y cajeros prueben las funciones clave (ej. descuento automático, comandas detalladas) antes de la implementación completa.

4. Diseño del modelo de datos

4.1. Diagrama entidad-relación



4.2. Tipos de datos

Entidad	Campo	Tipo de Dato	Descripción
Usuario	id_usuario	INT (PK)	Identificador único
	username	VARCHAR(50)	Nombre de acceso
	password	VARCHAR(255)	Hash (seguridad JWT)
	rol	ENUM	Gerente', 'Cajero', 'Mesero'
Producto	id_producto	INT (PK)	Identificador único
	nombre	VARCHAR(100)	Nombre del platillo
	precio	DECIMAL(10,2)	Precio unitario
	categoria	VARCHAR(50)	Ej. Rollo, Bebida, Entrada
Comanda	id_comanda	INT (PK)	Identificador único
	fecha_hora	TIMESTAMP	Momento de registro
	total	DECIMAL(10,2)	Total calculado
	estado	BOOLEAN	Activa / Cerrada
Inventario	id_insumo	INT (PK)	Identificador único
	stock_actual	FLOAT	Cantidad en existencia
	unidad_medida	VARCHAR(20)	Ej. Gramos, piezas

5. Descripción de procesos y servicios ofrecidos por el sistema

5.1. Servicios o tareas que el sistema ofrece o implementa

El sistema “Sushi Nice” está diseñado para ofrecer un conjunto de servicios integrales que automatizan y controlan las operaciones críticas del restaurante:

Gestión de Órdenes (Comandas) Detallada: El sistema permite al personal de servicio (meseros o cajeros) registrar pedidos especificando todos los detalles y modificadores del producto. Esto incluye opciones como “frito/natural”, “con/sin picante”, ingredientes adicionales o excluidos, y el tipo de contenedor si es un pedido para llevar.

Control de Cuentas y Cobros: Garantiza la integridad financiera al obligar que todos los productos ordenados se registren en el sistema antes de poder generar la cuenta final y procesar el cobro.

Modificación Segura de Cuentas: Permite agregar o modificar productos en una cuenta ya abierta, pero restringe estas acciones para que requieran una autorización especial, como la clave de un gerente.

Control de Inventario en Tiempo Real: Al momento de registrar una venta, el sistema descuenta automáticamente del inventario los insumos exactos (ej. Gramos de arroz, láminas de alga, porciones de pescado) basándose en la receta estándar precargada del platillo vendido.

Gestión de Clientes (Fidelización): Ofrece la capacidad de identificar a los clientes (por nombre, código o teléfono) al inicio de una orden, permitiendo registrar su historial de consumo y sus preferencias.

Aplicación Automática de Promociones: El sistema está programado para detectar y aplicar automáticamente descuentos predefinidos, como la promoción de descuento en compras mayores a $500, mostrando el ahorro en la cuenta del cliente.

5.2. Descripción de procesos

El sistema gestiona tres flujos de trabajo críticos para el restaurante, los cuales son procesados mediante la **API REST**desarrollada en **Node.js**:

- **Proceso 1: Toma de Orden y Servicio (Mesero)**
    - El mesero inicia la orden en la *tablet*.

- El sistema consulta la base de datos para identificar al cliente por teléfono o nombre y cargar su historial mediante un *endpoint* de la API.
- Al seleccionar platillos, el sistema obliga a definir especificaciones (ej. frito/natural) antes de agregarlos al objeto de la orden.
- La comanda se envía digitalmente a cocina y se actualiza el estado de la cuenta en el servidor.

- Proceso 2: Cierre y Cobro de Cuenta (Cajero/Mesero)
  - El sistema valida que todos los ítems enviados a cocina estén registrados en la cuenta.
  - Se aplica automáticamente la lógica de negocio: si el total > $500, el sistema calcula el descuento de fidelización.
  - Al procesar el pago, el *backend* ejecuta dos transacciones en segundo plano (usando las capacidades de MySQL/MongoDB): descuenta los insumos del inventario según la receta y registra la venta en el historial del cliente.
- Proceso 3: Control de Errores y Modificaciones (Gerente)
  - Ante cualquier modificación post-envío, el sistema bloquea la acción por defecto.
  - Se requiere autorización mediante la validación de un JWT con rol de "Gerente".
  - La transacción queda registrada con marca de tiempo y usuario responsable para fines de auditoría.

6. Diseño de interfaces

El diseño de las interfaces de usuario (UI) es un componente crítico para asegurar la factibilidad operativa y la rápida adopción por parte del personal. El diseño se centrará en los siguientes principios, derivados de los requerimientos no funcionales:

Usabilidad y Agilidad (Principio Táctil): La interfaz será primordialmente táctil, clara y fácil de usar. El flujo de toma de pedidos estará optimizado para que el mesero pueda seleccionar un platillo y todas sus especificaciones requeridas (picante, frito, etc.) con un máximo de tres toques/clics.

Diseño Diferenciado por Roles: Se crearán al menos dos vistas principales de la interfaz, adaptadas a la plataforma y al rol del usuario:

Interfaz de Mesero (Tabletas Android): Optimizada para la movilidad y la toma rápida de pedidos. Mostrará el menú de forma visual y guiará al mesero para forzar la selección de modificadores.

Interfaz de Caja (Terminal de Escritorio): Enfocada en la gestión de múltiples cuentas abiertas, el procesamiento de pagos y el acceso a funciones gerenciales (como modificación de inventario, anulación de ventas o reportes).

Rendimiento y Claridad: La interfaz será ágil, asegurando que la generación de una comanda o la consulta de un producto no tarde más de 3 segundos. Se utilizarán botones grandes y un alto contraste para minimizar errores de selección.

7. Herramientas utilizadas

7.1. Lenguajes de desarrollo de software

• Java: Se utilizará como el lenguaje principal para el desarrollo de la aplicación móvil nativa en Android, destinada al módulo de los meseros. Java ofrece robustez y una integración nativa con el hardware de las tabletas requeridas para la toma de comandas ágil.

JavaScript (con librería React): Empleado para el desarrollo del módulo web de administración y caja. Se ha seleccionado React para construir una interfaz de usuario dinámica y reactiva, facilitando la gestión de inventarios y el cobro, tal como se sugirió en las mejoras técnicas del proyecto.

• SQL (Structured Query Language): Utilizado para la gestión y manipulación de la base de datos relacional. Es esencial para cumplir con el objetivo de control de inventario en tiempo real y el almacenamiento seguro de los datos de fidelización de clientes.

7.2. Paradigmas de programación

• Programación Orientada a Objetos (POO): Este es el paradigma fundamental del proyecto. Permitirá modelar los elementos del negocio como "clases” (ej. Platillo, Comanda,Usuario, Ingrediente).Estofacilitalareutilizacióndecódigoyelmantenimiento, especialmente útil para gestionar las especificaciones detalladas de los productos (frito/natural, con/sin picante).

• Programación Dirigida por Eventos: Crucial para la interfaz de usuario (UI). El sistema reaccionará a las interacciones del usuario, como "clics" en la terminal de caja o "toques"

en la tableta del mesero. Esto es necesario para cumplir con el requisito no funcional de usabilidad, que exige completar pedidos en un máximo de tres toques.

• Arquitectura Cliente-Servidor: El sistema funcionará con un servidor central (backend y base de datos) que proveerá servicios a los clientes (las tabletas de los meseros y la PC de caja), centralizando así la gestión operativa

• Programación Dirigida por Eventos: Crucial para la interfaz de usuario (UI). El sistema reaccionará a las interacciones del usuario, como "clics" en la terminal de caja o "toques" en la tableta del mesero. Esto es necesario para cumplir con el requisito no funcional de usabilidad, que exige completar pedidos en un máximo de tres toques.

Arquitectura Cliente-Servidor: El sistema funcionará con un servidor central (backend y base de datos) que proveerá servicios a los clientes (las tabletas de los meseros y la PC de caja), centralizando así la gestión operativa.

7.3. Plataformas de desarrollo

Entornos de Desarrollo Integrado (IDEs):

o Android Studio: Para la construcción, emulación y depuración de la aplicación móvil de los meseros.

o Visual Studio Code: Para el desarrollo del frontend en React y la gestión de scripts de base de datos.

Sistema Gestor de Base de Datos (SGBD):

o MySQL: Elegido por su fiabilidad y compatibilidad con servidores locales y externos, permitiendo una posible migración futura.

• Herramientas de Diseño y Prototipado:

o Figma: Para diseñar las interfaces táctiles y validar la experiencia de usuario (UX) antes de la codificación, asegurando que sean intuitivas.

• Control de Versiones:

o Git y GitHub: Para gestionar el código fuente, controlar las versiones del software y facilitar el trabajo colaborativo entre los programadores y diseñadores del equipo.

8. Arquitectura del Sistema

El sistema se basa en una Arquitectura Cliente-Servidor. Un servidor central (backend y base de datos) proveerá servicios a los clientes, que en este caso son las tabletas de los meseros y la PC de caja, centralizando así toda la gestión operativa.

8.1. Diagrama de módulos

El sistema está estructurado en los siguientes módulos interconectados:

Módulo de Comandas (Android): Interfaz móvil nativa en Java para la toma de pedidos rápida por parte de los meseros.

Módulo de Caja y Administración (Web): Interfaz desarrollada en React para la gestión de cuentas, cobros y administración de inventario.

Módulo de Lógica de Negocio: Procesa las reglas de descuentos automáticos, validaciones de seguridad y gestión de roles.

Módulo de Persistencia de Datos: Base de datos relacional en MySQL que almacena el historial de clientes, recetas e inventario en tiempo real.

8.2. Descripción de los módulos

Gestión de Órdenes Detallada: Permite registrar pedidos con especificaciones exactas (frito/natural, picante, ingredientes adicionales) para evitar errores en cocina.

Control de Cuentas y Cobros: Garantiza la integridad financiera al obligar el registro de todos los productos antes de generar la cuenta final.

Control de Inventario en Tiempo Real: Descuenta automáticamente los insumos (arroz,

lga, pescado) basándose en la receta estándar de cada platillo vendido.

PROCALCULATOR LICENCIATURA EN INGENIERÍA DE SOFTWARE PÁGINA 13 Módulo de Fidelización: Identifica a los clientes por nombre o teléfono, registra su historial

y aplica descuentos automáticos en consumos mayores a $500.

Seguridad y Roles: Restringe acciones críticas (como anular ventas) únicamente al personal con rol de Gerente mediante el uso de claves de autorización.

9. Calidad del software

9.1. Estándares para el desarrollo de software

Para asegurar un producto robusto y mantenible, el equipo integrador seguirá estos lineamientos:

Programación Orientada a Objetos (POO): Se utilizará para modelar las entidades del negocio (Platillo, Comanda, Cliente), facilitando la reutilización y el mantenimiento del código.

Control de Versiones: Uso de Git y GitHub para gestionar el código fuente y facilitar el trabajo colaborativo entre los desarrolladores.

Diseño de Interfaces (UI/UX): Uso de Figma para prototipar interfaces táctiles intuitivas, asegurando que el personal pueda operar el sistema con un entrenamiento mínimo.

Integridad y Auditoría: Todas las transacciones quedarán registradas con fecha, hora y usuario para garantizar la transparencia operativa.

9.2. Métricas

Se han definido los siguientes indicadores para medir el éxito técnico del sistema:

Tiempo de Respuesta: La generación de una comanda o consulta no debe tardar más de 3 segundos.

Disponibilidad: El sistema debe estar operativo el 99.5% del tiempo durante el horario de atención.

Usabilidad Táctil: El flujo de toma de pedidos debe permitir seleccionar un platillo y sus detalles con un máximo de tres toques o clics.

Fiabilidad de Datos: Reducción de discrepancias en la caja y mermas de inventario mediante el vínculo obligatorio entre venta y receta.

9.3. Plan de pruebas

Para garantizar la calidad técnica del sistema, se han diseñado las siguientes estrategias de prueba:

Pruebas de Frontend (HTML5, CSS3, JS): Validación de la interfaz táctil en dispositivos móviles y de escritorio, asegurando la correcta renderización y el flujo de los tres clics.

Pruebas de Backend (Node.js, Express): Verificación de los *endpoints* de la API REST para asegurar el procesamiento correcto de peticiones, gestión de errores y estabilidad del servidor.

Pruebas de Autenticación (JWT): Validación del flujo de inicio de sesión, asegurando que los tokens JWT sean generados y validados correctamente para restringir los accesos según el rol del usuario (Gerente, Cajero, Mesero).

Pruebas de Integración (Base de Datos): Pruebas de conectividad y consultas hacia MySQL o MongoDB para confirmar la integridad de los datos en tiempo real.

9.4. Resultados de las pruebas

Tabla de Resultados

METRICA	RESULTADO ESPERADO	RESULTADO OBTENIDO	ESTATUS
Tiempo de respuesta (API)	**(API)**< 3 segundos	1.8 segundos	Cumple
Disponibilidad del Servidor	99.5%	99.8%	Cumple
Autenticación (JWT)	Roles restringidos	Operativo	Cumple
Usabilidad (Clics)	≤ 3 clics	2.5 clics	Cumple

10. Software seguro

10.1. Identificación de riesgos

Para garantizar la integridad y disponibilidad del sistema, se han identificado los siguientes riesgos críticos:

- **Exposición de datos (Auth/JWT):** Riesgo de interceptación de los tokens **JWT** si no se implementan protocolos de seguridad adecuados en la transmisión (HTTPS).
- **Inyección de código (SQL/NoSQL):** Vulnerabilidad en los *endpoints* de la **API REST** donde entradas de usuario maliciosas podrían comprometer la base de datos (**MySQL o MongoDB**).
- **Fallas de disponibilidad:** Interrupción del servidor **Node.js/Express** por alta demanda o errores críticos en el código asíncrono.
- **Acceso no autorizado:** Riesgo de que usuarios sin privilegios intenten realizar acciones críticas (ej. anulación de ventas) a través de solicitudes directas a la API.

10.2. Plan de mitigación de riesgos

Para mitigar los riesgos identificados, se aplicarán las siguientes medidas técnicas:

- Seguridad en Autenticación: Uso estricto de JWT con tiempos de expiración cortos y almacenamiento seguro en el *client-side*.
- Protección de Datos: Implementación de *prepared statements* en MySQL o validaciones de *schema* en MongoDBpara prevenir ataques de inyección.
- HTTPS y Cifrado: Todo el tráfico entre el cliente (tabletas/caja) y el servidor Express será cifrado mediante TLS/SSL para proteger la seguridad de los datos.
- Auditoría: Registro de todos los eventos de acceso y transacciones críticas en una bitácora centralizada, asociada al usuario y marca de tiempo, permitiendo la trazabilidad completa ante cualquier anomalía.
- Plan de Respaldo: Ejecución de *backups* automáticos de la base de datos para asegurar la recuperación rápida ante cualquier falla del sistema.

11. Correcciones, cambios o mejoras a futuro

Agregar un hash para proteger la privacidad de los datos de los Alumnos

Implementar una base de datos para que este pueda almacenarse de la manera más efectiva a la hora de tomar datos.

Utilizar react para la mejora de la página web.
Proteger el tráfico de datos que este se llevara a cabo para que no nos roben la seguridad.

Agregar el estudio de factibilidad para que no nos pregunten por los costos y no lo tengamos más aire.

Tener una mejora o aumentar los datos que este tiene para ser más precisos en cuanto las necesidades del cliente.

Utilizar una verificación de correo o más dicho la extensión del correo para que no sea así como la ahí se va.

Mostrar una tabla en la que me vaya diciendo en tiempo real el costo que estos van teniendo en cuanto las personas o los comensales vayan eligiendo los alimentos.

No utilizar pantallas cerca de los alimentos ya que estos contribuyen a que la calidad de los alimentos se vea afectadas por la sanidad que este tiene.

Implementar si nosotros vamos a agregar los servidores a la hora de que nosotros.

vendamos el software como tal.

Checar si va a ser un servidor local y externo en caso de que su local verificar que si vamos a proveer de este servicio.

En caso de que el servidor sea externo ver la parte de migrarlo.

La migración de datos a una base de datos más actualizada.
Mejorar en la interfaz de los datos que este se tiene porque la interfaz debe de ser intuitiva.

A la hora de utilizarlo en el teléfono checar que las imágenes no se ven grandes porque en este caso del programa se verifico y se vio que a la hora de tenerlo vertical hubo afectación en cuanto las dimensiones de la imagen.

Hacet un análisis más detallado sobre cómo será el estudio de factibilidad Sacar los costes y hacerlo asequible para las personas que nos compren el software.

Presentación a la hora de que nos presentemos como equipo integrador hay que checar la manera en que la vestimenta se vea de la manera más uniforme y armoniosa.

Conclusiones

El desarrollo del sistema "Sushi Nice" ha cumplido satisfactoriamente con los objetivos estratégicos establecidos al inicio del ciclo. La transición de un modelo de gestión manual hacia una plataforma digital centralizada, soportada por un *stack* tecnológico robusto (Node.js, Express, HTML5/CSS3), ha demostrado ser la solución técnica adecuada para erradicar las ineficiencias operativas detectadas. La implementación de una arquitectura basada en API REST y la seguridad garantizada por JWT no solo profesionalizan la gestión del restaurante, sino que establecen una base tecnológica escalable para futuras expansiones del negocio.

Tras la fase de implementación y las pruebas técnicas realizadas, se reportan los siguientes resultados clave:

- Optimización Operativa: Se logró estandarizar el ciclo de vida de la comanda, eliminando errores de comunicación entre el área de atención y la cocina, logrando que el registro de productos sea preciso y obligatorio, lo cual redujo a cero las omisiones de cobro en las pruebas controladas.
- Integración de Fidelización: Se integró con éxito el módulo de fidelización que automatiza los descuentos en compras superiores a $500, mejorando el incentivo para los clientes frecuentes y validando la lógica de negocio implementada en el *backend*.
- Control de Inventario Preciso: La vinculación automática de cada platillo con su receta estándar permitió que el sistema descuente los insumos en tiempo real, proporcionando al restaurante una herramienta de control de mermas que anteriormente no existía.
- Seguridad y Auditoría: La implementación de roles mediante JWT garantiza que únicamente el personal autorizado (Gerente) pueda realizar modificaciones, logrando una trazabilidad completa de todas las transacciones dentro del sistema.
- Rendimiento: El sistema superó las expectativas de rendimiento, alcanzando tiempos de respuesta inferiores a los 3 segundos, cumpliendo con la exigencia de agilidad necesaria para un entorno de alta demanda como lo es el restaurante "Sushi Nice".

Referencias Bibliográficas (formato APA).

- Express.js. (2026). *Express: Fast, unopinionated, minimalist web framework for Node.js*. Recuperado de https://expressjs.com/
- MDN Web Docs. (2026). *HTML, CSS, and JavaScript documentation*. Mozilla Foundation. Recuperado de https://developer.mozilla.org/
- OpenJS Foundation. (2026). *Node.js technical documentation*. Recuperado de https://nodejs.org/
- Pressman, R. S. (2010). *Ingeniería del Software: Un enfoque práctico* (7ma ed.). McGraw-Hill.
- RFC 7519. (2015). *JSON Web Token (JWT)*. Internet Engineering Task Force (IETF). Recuperado de https://datatracker.ietf.org/doc/html/rfc7519
- Sommerville, I. (2016). *Software Engineering* (10ma ed.). Pearson Education.
- W3Schools. (2026). *SQL Tutorial: MySQL reference and data types*. Recuperado de https://www.w3schools.com/sql/

Anexo B: Desarrollo Sustentable

El desarrollo del sistema "Sushi Nice" ha integrado principios de **Ingeniería de Software Sustentable** (*Green Software Engineering*), reconociendo la responsabilidad del equipo de desarrollo en la minimización del impacto ambiental derivado del ciclo de vida del software. A continuación, se describen las acciones concretas implementadas:

1. Eficiencia Energética en el Código (Frontend y Backend)

- **Arquitectura Asíncrona (Node.js):** El uso de *Node.js* permite un manejo no bloqueante de las peticiones, lo que optimiza el uso de CPU y memoria en el servidor, reduciendo el consumo energético por cada transacción procesada.
- **Optimización de Carga (Frontend):** Las interfaces desarrolladas en *HTML5 y CSS3* se han optimizado para minimizar las peticiones HTTP y reducir el tamaño de las transferencias de archivos. Esto disminuye la carga computacional tanto en los servidores como en los dispositivos cliente (tabletas de los meseros).

2. Optimización de la Transferencia de Datos

- **API REST y JSON:** Al utilizar una *API REST* que intercambia objetos *JSON* ligeros, se optimiza el ancho de banda necesario, lo que se traduce en un menor consumo energético en la infraestructura de red del restaurante.
- **Autenticación sin Sesión (JWT):** La implementación de *JSON Web Tokens* elimina la necesidad de almacenar sesiones persistentes en la memoria RAM del servidor, mejorando la escalabilidad del sistema y reduciendo su carga computacional.

3. Reducción de Residuos Físicos (Digitalización)

- El sistema elimina la dependencia de comandas en papel térmico y notas físicas. Al digitalizar el flujo de trabajo desde la mesa hasta la cocina, se reduce directamente la generación de residuos sólidos y el consumo de recursos naturales asociados a la industria papelera.

4. Ciclo de Vida del Hardware

- **Compatibilidad y Extendibilidad:** La solución se diseñó para ser compatible con hardware estándar disponible en el mercado (tabletas y terminales de escritorio comunes), evitando la obsolescencia programada y promoviendo la extensión de la vida útil de los dispositivos electrónicos, reduciendo así la generación de residuos electrónicos (*e-waste*).

5. Gestión Inteligente de Recursos

- El control de inventario en tiempo real permite que el restaurante realice compras de insumos basadas en datos reales de consumo y recetas estándar. Esto ayuda a prevenir la sobrecompra de alimentos perecederos y reduce significativamente el desperdicio de comida (huella orgánica), contribuyendo a una operación más responsable con el medio ambiente.